XRPL COMMONS HACKATHON NYC · 24–25 OCT 2026

Settle real assets. Pay only when the deal is done.

LiquidSettle is a settlement engine on the XRP Ledger: two token escrows lock the money and the title, the real world (USPS, a vault) is anchored on the ledger, and each leg releases only on proof. Every step of every deal is a transaction or a labelled simulation. This document walks the five workflows, step by step.

HACKATHON · THE ENTRY TO JUDGE

How to read the labels

HACKATHON	The entry to judge: the LiquidSettle settlement engine (settle/, the deals API in openapi/).
DEMO	Shown for context, not entered: the Lending desk (dashboard/) and CollateralLock, our XLS-66 amendment on our own devnet.
LIVE · XRPL TESTNET	A transaction the engine submits to the public XRPL Testnet; its hash opens on any Testnet explorer.
OWN NODE · DEVNET	A transaction on our XLS-66 devnet (network_id 424). Part of the DEMO; hashes exist only there.
SIMULATED	Off the ledger and labelled so on every screen: ACH and card payments.
SAMPLE	The repository's sample data (?mock=1). Recorded on an in-memory ledger; its hashes are not proofs.

What the judges look at

HACKATHON

The settlement engine is the entry. Everything else in the repository is DEMO — shown for context.

PART	WHERE	TAG	WHAT IT IS
Settlement engine (desk, deals, vault, money, activity, ways out, workflow console)	`settle/`	HACKATHON	The front end and its deals API — the six Figma frames, built and wired to the ledger.
Deals API (LiquidGate /api/lg/v1/deals…)	`openapi/ · sdk/`	HACKATHON	The contract the engine speaks; the platform behind it stays private.
Lending desk (the eleven beats)	`dashboard/`	DEMO	An earlier operator console; not entered.
CollateralLock (XLS-66 amendment) + own devnet	`spec/ · docs/collateral-lock/`	DEMO	The loan workflow runs on it; its evidence and hash register are attached as context.

Try it in one minute: `scripts/serve.sh 8788 settle` → `http://localhost:8788/?mock=1` (sample data, labelled SAMPLE). Against a platform, the same screens run every step on the ledger.

Honest scope: the engine's transactions are built for the public XRPL Testnet; the hash register (p. 9) lists only hashes read back from a ledger.

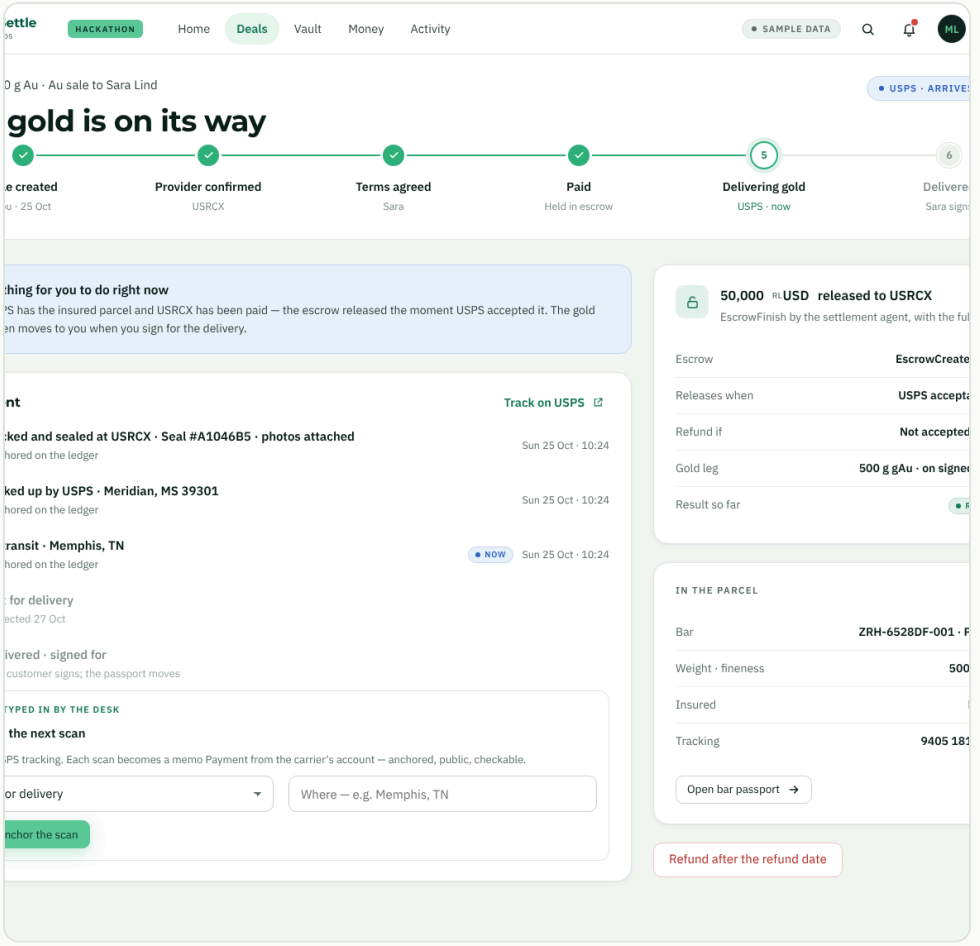
A · Buy and deliver — delivery versus payment

Two token escrows on the XRPL Testnet: the money and the title each wait for their own proof. Nobody can move either early; both refund after twelve days.

HACKATHON

XRPL TESTNET

#	WHO	TRANSACTION	LEDGER	WHAT IT DOES	EXPECTED
1	DESK	Payment (MPT)	TESTNET	**Provider confirmed.** The issuer mints the bars to the desk's inventory; serials in the memo.	tesSUCCESS
2	CUSTOMER	— (price hold)	SIMULATED	**Terms agreed.** The Au price is held two minutes; the customer picks how to pay.	
3	DESK	EscrowCreate (MPT)	TESTNET	**Title leg.** The Au token, desk → customer, PREIMAGE-SHA-256 condition, CancelAfter 12 days (XLS-85).	tesSUCCESS
4	CUSTOMER	EscrowCreate (IOU)	TESTNET	**Payment leg.** RL USD, customer → desk, its own condition and the same refund date.	tesSUCCESS
5	USPS · BY HAND	Payment (memo)	TESTNET	**Picked up.** The scan is anchored by the logistics role (memo liquidsettle/usps).	tesSUCCESS
6	AGENT	EscrowFinish	TESTNET	**Payment released** to the desk on *picked up*. The fulfillment comes sealed from the platform; it is never shown.	tesSUCCESS
7	USPS · BY HAND	Payment (memo)	TESTNET	**Delivered** — signature at the verified address, anchored like every scan.	tesSUCCESS
8	AGENT	EscrowFinish	TESTNET	**Title released:** the Au token reaches the customer. The deal settles; the receipt lists every hash.	tesSUCCESS



LS-1046 · with USPS (sample data)

B · A refusal is a result: tecNO_AUTH → credential → pay

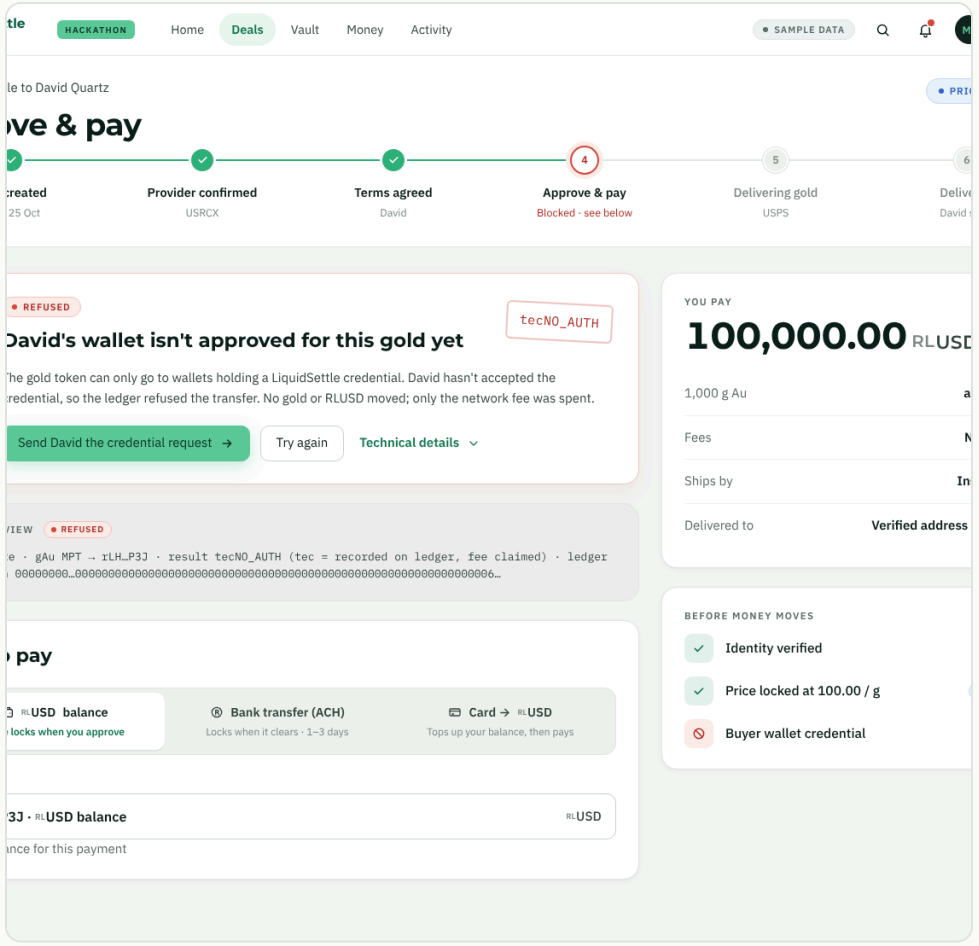
The Au token requires authorisation. A wallet without the LiquidSettle credential is refused by the ledger itself — the engine records the refusal with its hash and offers the way through.

HACKATHON

XRPL TESTNET

#	WHO	TRANSACTION	LEDGER	WHAT IT DOES	EXPECTED
1	CUSTOMER	EscrowCreate (MPT → customer)	TESTNET	The title leg to a wallet the issuer has not authorised. The token requires authorisation (RequireAuth).	tecNO_AUTH
2	ENGINE	— (state, not an error)	SIMULATED	The deal stops at step 4 as **refused**: the refusal card, the technical view with the hash. No money moved; only the fee.	
3	DESK	CredentialCreate	TESTNET	**LiquidSettle.KYC** issued to the customer's wallet (XLS-70).	tesSUCCESS
4	CUSTOMER	CredentialAccept	TESTNET	The customer accepts it: the credential is now on their account.	tesSUCCESS
5	DESK	MPTokenAuthorize (holder)	TESTNET	The issuer reads the accepted credential and authorises the holder for the Au token.	tesSUCCESS
6	CUSTOMER	EscrowCreate × 2	TESTNET	**Approve & pay** again: both legs lock (the DvP above continues).	tesSUCCESS

> Not yet confirmed on the public Testnet from this build: that the token escrow to an unauthorised destination returns **tecNO_AUTH**. If the ledger accepts it, the deal proceeds and says so.



LS-1045 · Approve & pay, refused (sample data)

C · USPS by hand — every scan is a ledger entry

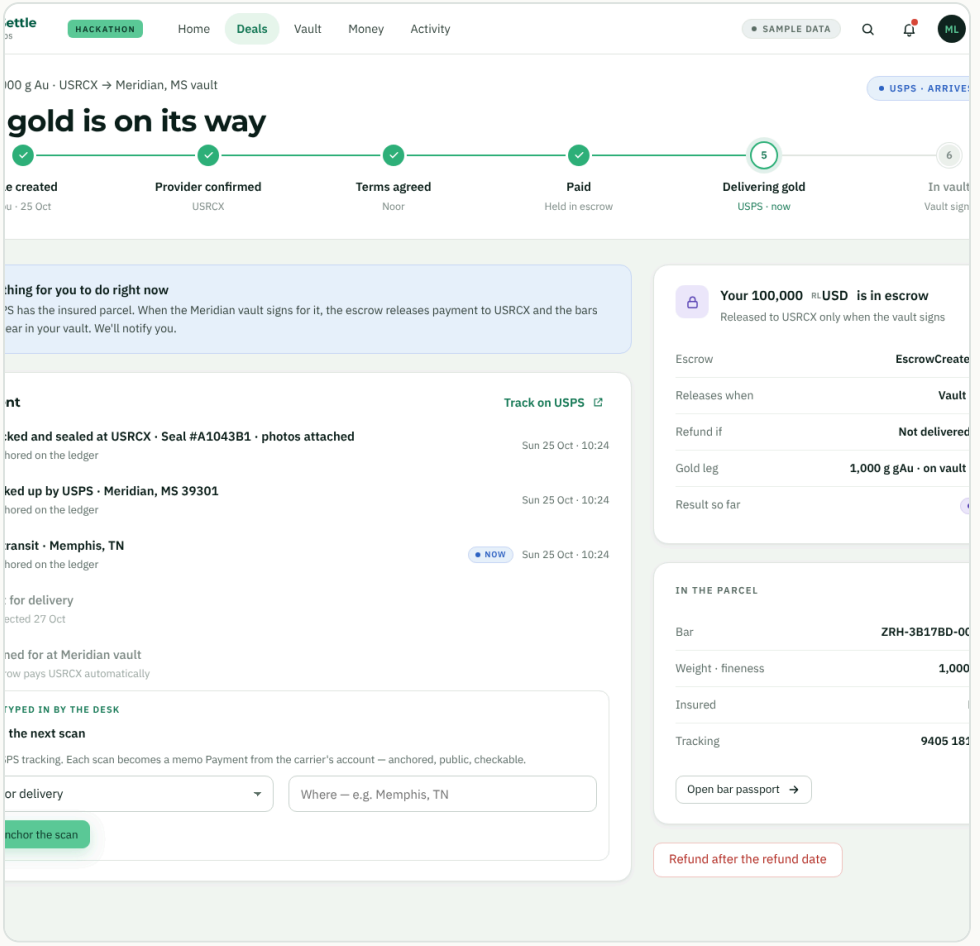
HACKATHON

XRPL TESTNET

We type the tracking scans ourselves (code · place · time). Each one is anchored by the logistics role as a memo Payment and drives the release rules. No call goes to the USPS API.

CODE	MEANING	WHAT IT RELEASES
`picked_up`	USPS took custody	Delivery: releases the payment leg to the desk
`in_transit`	Moving through the network	—
`out_for_delivery`	On the truck	—
`delivered`	Signed for at the verified address	Delivery: releases the title leg → settled
`vault_signed`	The vault signed for the parcel	Store / store & borrow: releases both legs (store → settled; loan → step 5)
`exception`	Damaged, held, returned	Nothing releases; the refund date still protects both sides

> The anchor: `Payment` of 1 drop, logistics → desk, memo `liquidsettle/usps` {deal, code, where, at, tracking} — the timeline on the deal and the receipt carry its hash.



LS-1043 · Your Au is on its way (sample data)

D · Store in the vault, refund, bank transfer

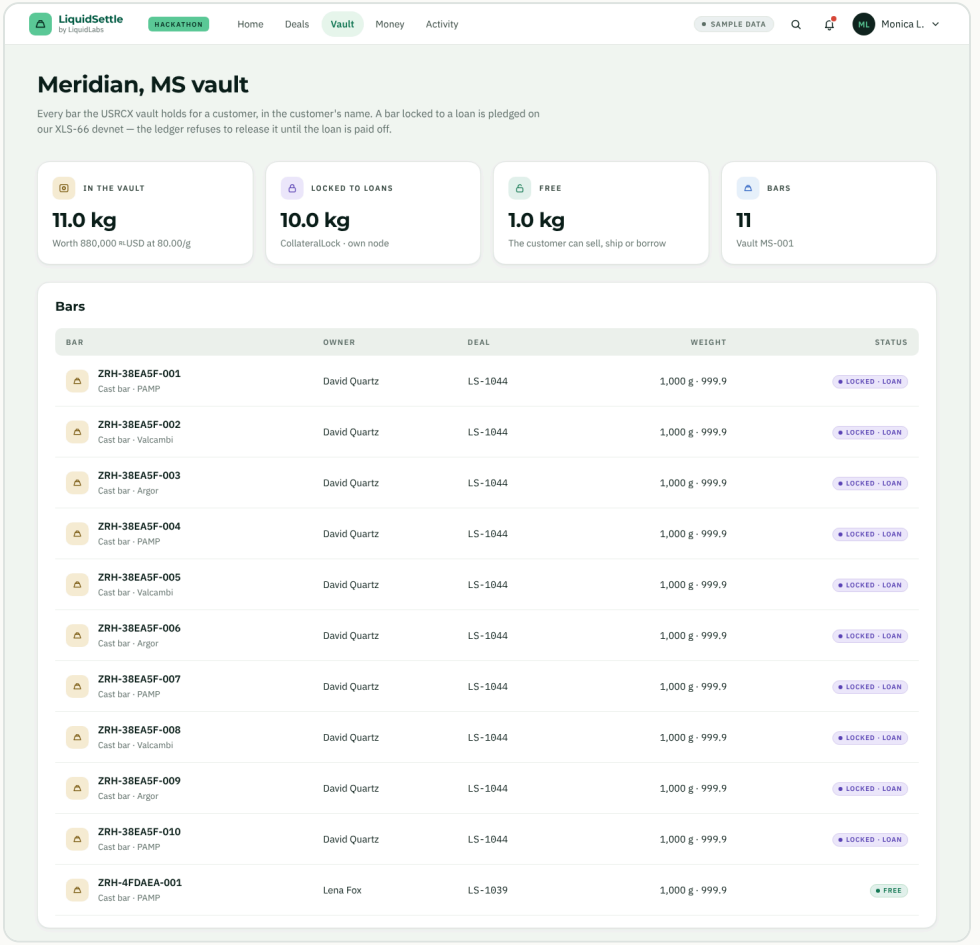
The same escrows with another release rule: the vault's signature releases both legs. A refund needs nobody's consent after the date. A bank transfer is simulated and says so.

HACKATHON

XRPL TESTNET

ACH SIMULATED

#	WHO	TRANSACTION	LEDGER	WHAT IT DOES	EXPECTED
1	DESK	EscrowCreate × 2	TESTNET	Buy & store: the same two legs, the destination is the customer's vault position.	tesSUCCESS
2	VAULT · BY HAND	Payment (memo)	TESTNET	**Vault signed** — anchored (memo liquidsettle/vault).	tesSUCCESS
3	AGENT	EscrowFinish × 2	TESTNET	Both legs release at once; the bars are listed in the customer's name.	tesSUCCESS
4	AGENT	EscrowCancel × 2	TESTNET	**Refund** (LS-1037): nothing arrived by the refund date → both legs go back. Offered only after CancelAfter.	tesSUCCESS
5	CUSTOMER	— (ACH)	SIMULATED	**Bank transfer** (LS-1048): the price locks when it clears (1–3 days). Simulated and labelled.	



Vault (sample data)

E · Store and borrow — the loan workflow

The engine's loan journey (HACKATHON) runs its ledger steps on our XLS-66 devnet with CollateralLock (DEMO): the pledged Au cannot leave while the loan is owed.

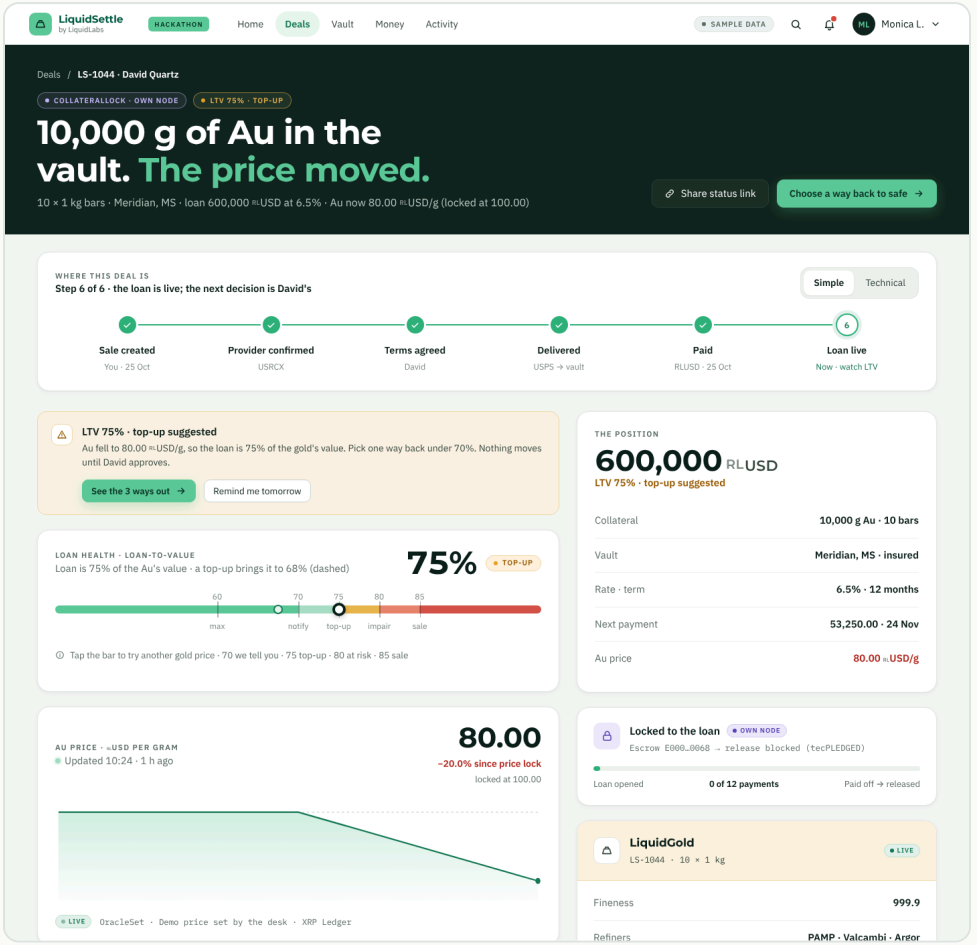
HACKATHON · WORKFLOW

DEMO · NETWORK

OWN NODE

#	WHO	TRANSACTION	LEDGER	WHAT IT DOES	EXPECTED
1	CUSTOMER	EscrowCreate (LGAU)	OWN NODE	**The pledge.** 10,000 g into a token escrow, Account = Destination = the borrower.	tesSUCCESS
2	CUSTOMER + LENDER	LoanSet (two signatures)	OWN NODE	The loan with **CollateralID** = the pledge escrow; the borrower signs, the broker countersigns.	tesSUCCESS
3	LEDGER (PROBE)	EscrowFinish	OWN NODE	The engine proves the lock: releasing the pledge while the loan is owed is refused.	tecPLEDGED
4	ORACLE	OracleSet	TESTNET	Au 100 → 80 per gram: the loan is re-measured, LTV 75 % → **top-up**.	tesSUCCESS
5	CUSTOMER	LoanPay (overpayment)	OWN NODE	**Ways out · pay down** 60,000: back under 70 %.	tesSUCCESS
6	CUSTOMER	EscrowCreate / Payment	OWN NODE	**Add Au** (a second pledge) or **buy more** (pay, then pledge).	tesSUCCESS
7	CUSTOMER	LoanPay (full)	OWN NODE	**Repay**: the loan is paid off.	tesSUCCESS
8	CUSTOMER	EscrowFinish	OWN NODE	The same release now succeeds: the pledge returns to the borrower.	tesSUCCESS

> The devnet and the amendment are DEMO: not on any public network. Their evidence (patches, 7 cases / 687 checks, 23/23 two-validator agreement) is in `docs/collateral-lock/`.



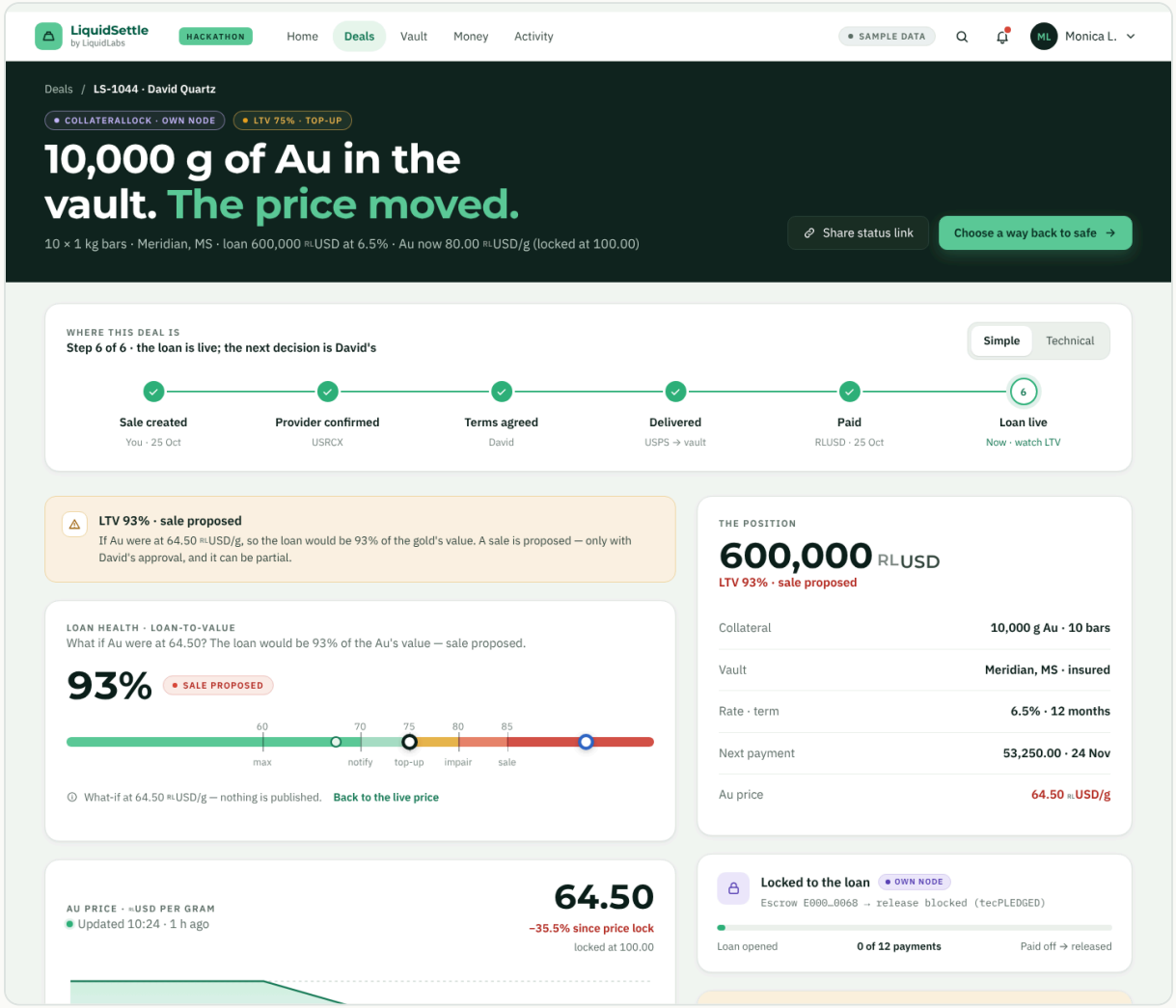
LS-1044 · the loan live (sample data)

The Au price moves — everything follows

The AI price agent publishes OracleSet (XAU/USD). Drag the loan-health meter for a what-if: the rung, the ways out and the chart move; nothing is published until an operator says so.

HACKATHON

ORACLESET · TESTNET



What-if at 64.50 per gram: LTV 93 % · sale proposed — nothing published (sample data)

The ladder

LTV	RUNG	WHAT HAPPENS
≤ 60 %	max	The most a new loan may be.
70 %	notify	The customer is told.
75 %	top-up	Pick a way back under 70 %.
80 %	impair	The loan is flagged.
85 %	sale	A sale is proposed — only with approval, can be partial.

Ways back to safe

- Pay down — LoanPay (overpayment)
- Add Au — a second pledge
- Buy more — pay, then pledge
- Repay in full — LoanPay, then the release

Hash register · settlement engine

HACKATHON

09

Every row was read back from a ledger by export_hashes.py: Testnet rows open on any Testnet explorer; own-node rows exist only on our devnet (DEMO).

Pending — no settlement-engine hash is printed until it is read from a ledger

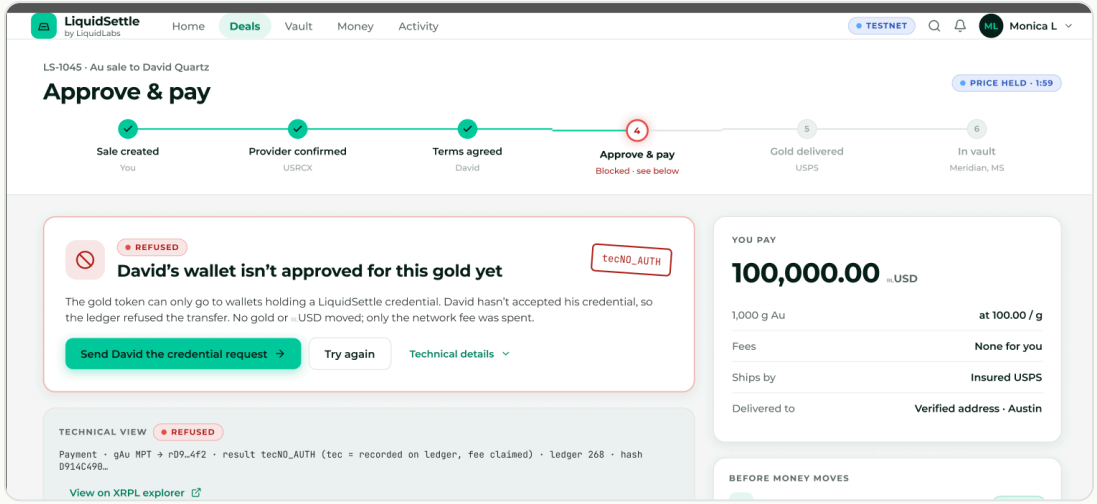
The engine was built and tested this weekend against an in-memory ledger; its sample data is labelled SAMPLE and links to no explorer. To fill this page: on a platform, press **W → Make the demo workspace** (about ten minutes of real Testnet and devnet transactions), then `python3 docs/build/export_hashes.py https://<platform>/api/lg/v1 <session token>` and rebuild this PDF.

> The hash register that exists today is CollateralLock's (DEMO): source, test logs, own-node run and the two-validator crosscheck — `docs/collateral-lock/CollateralLock_Evidence_and_Hashes.pdf`, pages 3, 8, 9, 11, 14 and 15.

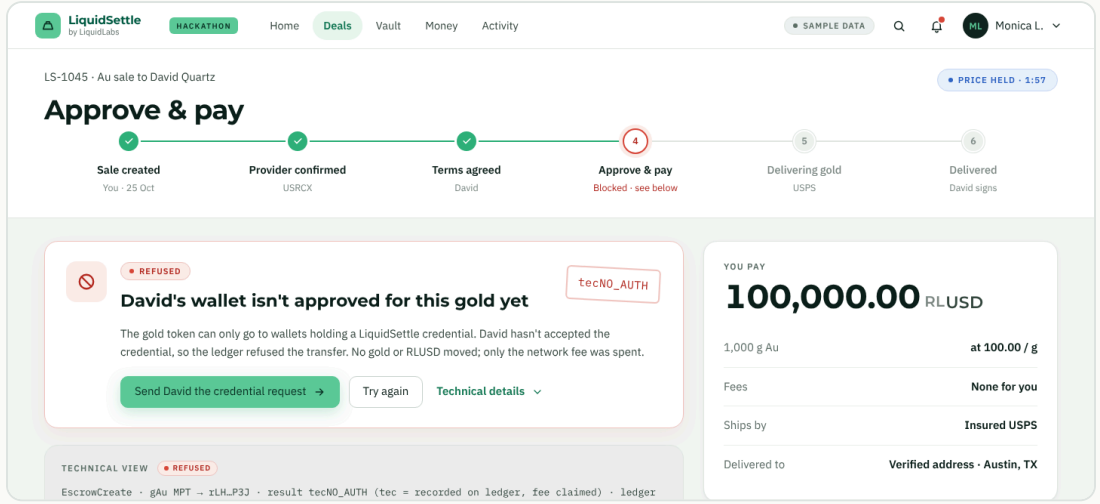
From the Figma frames to the engine

HACKATHON

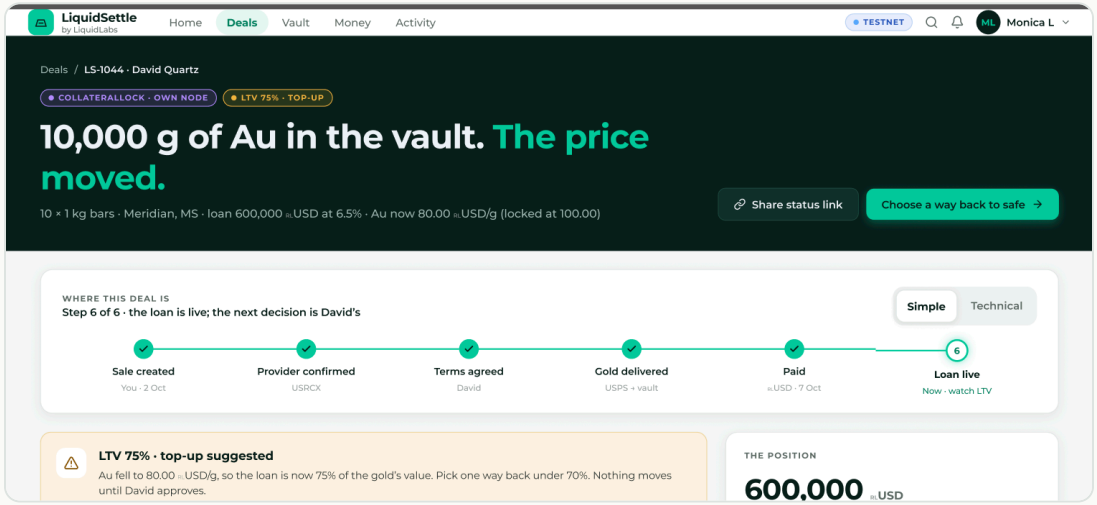
The six frames were mapped one to one: the desk, the New deal modal, Approve & pay refused, on its way, the live loan, ways out.



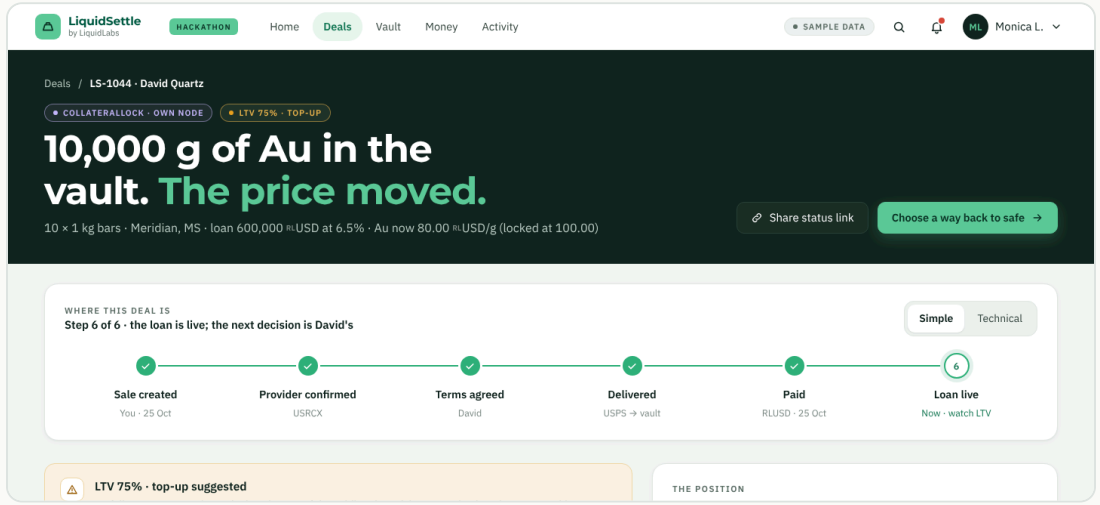
Figma · Approve & pay



Engine · Approve & pay



Figma · Loan live



Engine · Loan live

What is real, what is simulated, what is not yet verified

HACKATHON

Said plainly, as on every screen.

ON THE LEDGER

- Both escrow legs, their finish and cancel
- KYC credential: create, accept, authorise
- Every USPS and vault scan (anchor)
- The Au price (OracleSet)
- Loan: pledge, LoanSet, probe, LoanPay, release — on our devnet (DEMO)

SIMULATED, LABELLED

- ACH and card payments
- USPS itself: scans are typed in by the desk
- RL USD is the desk's own demo issuer on a test network
- The repository's sample data (SAMPLE)

NOT YET VERIFIED

- tecNO_AUTH for a token escrow to an unauthorised wallet on the public Testnet
- An RL USD vault on the devnet (falls back to XRP)
- The LoanPay flags on our devnet build
- The settlement-engine hash register (p. 9)

Security: no seed, key or node URL is in this repository; the platform signs with custody-vault keys and refuses mainnet outside production. `scripts/check.py` enforces it on every push.